

Usability engineering and user experience

Authors

Matt ter Steege
Matttersteege@gmail.com

Contents

1	Wat is UE en UX	2
1.1	Human-centred design	2
2	Usability	2
3	Usability engineering	2
3.1	Usability inspection	3
3.2	Usability testing	3
3.3	Geautomatiseerde testen	3
4	Performance, issue-based & selfreported metrics	4
4.1	Performantiemetrieken	4
4.2	Zelfgerapporteerde metrieken	5

1 | Wat is UE en UX

Het houdt zich bezig met interactieve systemen en hoe deze goed kunnen interacteren met gebruikers. Voorbeelden van een interactief systeem is bijvoorbeeld een telefoon, website, oid.

Human-centred design

Dit concept gaat erom dat je een alles zo makkelijk te gebruiken mogelijk maakt. Het voorbeeld van de afstandsbediening voor de TV. Er zitten heel veel knoppen op, om dit gebruiksvriendelijker te maken kan je dus een groot deel schrappen om alleen de essentiële knoppen over te houden.

2 | Usability

“Extend to which a system, product or service can be used by specified users to achieve specified goals with effectiveness, efficiency and satisfaction in a specified context of use”

- Effectiveness: Bereiken gebruikers het gewenste doel?
- Efficiency: is de nodige inspanning zo klein mogelijk?
- Satisfaction: vinden gebruikers het product prettig in gebruik?
- Utility: Heeft het systeem de juiste functionaliteit
- Safety: Hoe (be)veilig(d) is het gebruik, dit gaat niet over de impact van de actie. (chatGPT is veilig, maar de impact van het antwoord misschien niet)
- Learnability: Is het gebruik van het systeem makkelijk én snel te leren
- Memorability: Is het gebruik van het systeem makkelijk te onthouden.

Voorbeeld (**Effectiveness, Efficiency en Satisfaction**):

Je fiets een zo kort mogelijke route van A naar B maar deze route is niet leuk. **Effectiveness:** Ja, **Efficiency:** Ja, **Satisfaction:** Nee

Je fiets ietsje om, maar de route is veel leuker. **Effectiveness:** Ja, **Efficiency:** Nee, **Satisfaction:** Ja

3 | Usability engineering

“Specifying quantifiable measures of product performance documenting them in a usability specification, and assessing the product against them”

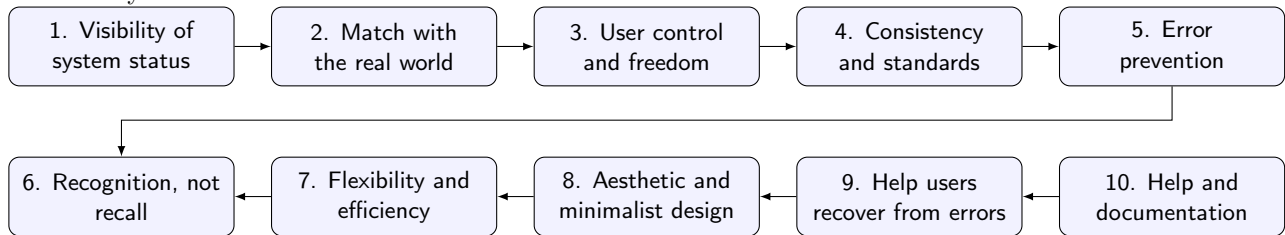
Je moet dus gaan vastleggen welke doelen een product heeft en evalueren of deze gehaald worden. Dus gebeurd iteratief, dus door te houden wat er goed is, en aanpassen wat een **usabilityprobleem** is (verwarrend, slecht te gebruiken, etc.). Dit kan je doen aan de hand van verschillende metrieken. Denk aan een metriek voor learnability: de tijd nodig om een taak te leren. Of Memorability: het aantal fouten dat wordt gemaakt nadat een product een week niet gebruikt werd. Deze metrieken zijn duidelijk gedefinieerd, direct of indirect meetbaar, kwantificeerbaar én representeren de ervaring van de gebruikers.

Om te testen wat er goed gaat aan je ontwerp kan je vroeg in het ontwerp “formatief testen”, hier test je je Low-fidelity prototype. Het handige hieraan is dat je dus heel snel veel verschillende ontwerpen kan , maar het nadeel is dat het nog niet helemaal geweldig is uitgewerkt, en dus wel nog een ruwe schets is. Het is ook heel goedkoop. Een laatste idee is dat mensen veel eerlijker zijn en *kritischer feedback geven*. Nadeel is dat het wel quick & dirty is, het blijft een ruwe schets

Als je midden in ontwikkeling wilt gaan testen, dan kan je je high-fidelity prototype “summatief testen”. Het is eigenlijk het omgekeerde van het formatief testen. Een ander pluspunt is dat je daadwerkelijk testen kan gaan uitvoeren en kwantitatieve vergelijkingen kan doen.

Usability inspection

Bij usability inspection ga je interface-experts laten kijken naar je high-fidelity prototypes of bestaande applicaties op basis van algemene regels voor goed interface ontwerp. Je gaat de belangrijkste usability-problemen oplossen zoals verwarrende verwoordingen, inconsistente layout, fout kleurgebruik, etc. De top tien *vuistregels* van usability heuristics.



Hoe gaat zo'n inspectie in de praktijk:

1. Afstemming tussen experts
2. Daadwerkelijk evaluatie
 - (a) Eerste de applicatie doorlopen om vertrouwd te raken
 - (b) De applicatie nogmaals doorlopen om op specifieke aspecten te richten
3. Heuristieken evalueren en prioriteiten toewijzen aan usabilityproblemen

Voordelen hiervan zijn dat het redelijk makkelijk is om een checklist af te gaan en de experts weten wat ze doen (hopelijk, een keerzijde is wel dat het geld kost, en dat ze moeilijk te bereiken zijn, ook kan het zijn dat alleen globale problemen worden gevonden, maar specifieke problemen worden niet gevonden).

Walktroughs

- **Cognitive walktrough:** Ontwikkelaars kiezen een applicatie ter evaluatie en maken een gebruiksscenario. Experts krijgen informatie of de context en taakdetails.
- **Pluralistic walktrough:** eindgebruikers, ontwikkelaars en usability-experts doorlopen samen een specifiek scenario en bespreken usability-problemen

Usability testing

Bij usability testing gaan representatieve testpersonen toetsen of ze als eindgebruikers door de applicatie kunnen navigeren.

Je hebt een standaard dopmaak van een usability-test

- **Verwelkoming:** Deelnemers maken kort kennis
- **Briefing:** introduceer de applicatie, vraag toestemming, leg de procedure uit. En zeg iets rond de lijnen: *"Ik ga jou niet testen, maar de applicatie"*.
- **Uitvoering:** De deelnemers voeren de taken uit. Think aloud study oid.
- **Afsluiting:** Beantwoord extra vragen, bedank deelnemer(s)

Je hoeft eigenlijk maximaal 5 verschillende deelnemers te testen, want daarna heb je al 85% van de problemen gevonden, daarna leer je eigenlijk niet heel veel meer. Maar het kan natuurlijk zo zijn dat er veel meer problemen zijn, dus het hangt ook af van tijd en geld. Eigenlijk moet je stoppen op het moment dat er geen nieuwe problemen gevonden worden. Maar welke taken geef je de gebruiker?

- De taken moeten een duidelijk eindpunt hebben.
- Taken moeten de belangrijkste functionaliteiten van de applicatie testen
- Taken die kennis opzoeken omvat, mag dat niet afhangen van voorkennis

Maar wanneer ga je ingrijpen? Dat kan na een bepaald tijdslimiet, of als er duidelijk is als een gebruiker echt helemaal niet meer ziet wat diegene moet doen. Dan komt er niks nuttigs meer uit, dus dan haal je geen kennis meer uit de testpersoon. Stel wel een minimumtijd in, dus ga pas helpen na 30 seconden oid.

De antwoorden kan je dan als transcript, notitie oid. opschrijven en analyseren.

Geautomatiseerde testen

Geen testpersonen

4 | Performance, issue-based & selfreported metrics

Performantiemetrieken

“In plaats van werken met subjectieve problemen en severity rates leg je concrete taken of doelen vast en meet de performantie van mensen die een applicatie gebruiken.”

Aanname is wel dat het gebruikersgedrag wat je wil meten echt daadwerkelijk kan meten.

Performantiemetrieken zijn bedoeld om wat-vragen te beantwoorden: “wat zijn de tijdskosten” ed.

Er zijn verschillende performantiemetrieken, dat zijn:

Succesvol voltooide taken (Meest voorkomen)

Je hebt een taak met een concreet einddoel en een definitie van “succesvol”. Voorbeeld: *Zoek op website van de UU welke docent(en) vak x geeft*. Dit kan je **binair** meten, dit is of wel of niet succes (pass of fail).

Een fail is bijvoorbeeld: opgeven van deelnemer, te veel tijd nodig, de taak fout uitvoeren of de onderzoeker onderbreekt de deelnemer.

Dit kan je dan rapporteren als een grote tabel waar je dan deelnemer uitzet tegen de taken met een 1/0 waarde. Je moet je wel bewust zijn van de context en of verschillende groepen invloed kunnen hebben op de uitkomst. Misschien zijn mensen met een tech achtergrond veel succesvoller dan mensen in de geneeskunde achtergrond.

Maar dit heeft ook zijn problemen, want een taak kan ook gedeeltelijk correct zijn en dan is ja/nee veel te onnauwkeurig. Dus dan kan je correctheid op basis van 3 of 4 niveau's uitzetten.

Tijd om taken te voltooien

Tijd hoeft niet altijd een goede metriek te zijn, als het doel is om mensen te entertainen, dan wil je tijd niet perse verlagen. Wil je dat taken sneller uitgevoerd kunnen worden (in bijvoorbeeld productiviteit apps) dan wil je dat dit sneller gebeurt. Het ligt ook aan wat de gebruikers eigenlijk willen.

Maar wat als een taak niet succesvol is uitgevoerd? Dan kan je dat opsplits in 2 soorten, verkeerd antwoord en afgebroken. Als een taak niet succesvol is afgerond dan wel erbij meten met de notitie dat het niet correct is afgerond, als je de taak afbreekt (of ze zelf afbreken) dan moet je de tijd niet meten.

Fouten

Fouten zijn meestal het gevolg van usability-problemen. Dingen zoals een onduidelijke positie van een knop oid kan ervoor zorgen dat een actie op het verkeerde element wordt uitgevoerd. Andere soorten zijn bijvoorbeeld ook incorrecte data.

Je kan het aantal fouten analyseren door een gemiddelde van het aantal fouten pp te berekenen. Het percentage dat fouten maakt op een bepaalde taak. Niveau van ernst aan de fouten toekennen (hoog, gemiddeld, laag) of terugkerende fouten identificeren.

Efficiënte

“Inspanning nodig om een taak te voltooien”

- **Cognitieve inspanning:** bepalen wanneer welke actie uitgevoerd moet worden en het resultaat interpreteren.
- **Fysieke inspanning:** muis bewegen, toetsen indrukken etc.

Je kan efficiënte uitdrukken als een combinatie van zo weinig mogelijk dezelfde pagina's bezoeken, of het kortste pad naar een specifieke webpagina vinden.

Leerbaarheid

Leerbaarheid is een maat voor hoe gemakkelijk een applicatie geleerd kan worden. Het leren van een applicatie kost tijd, dus niet alleen de eerste indruk telt. Je wilt ook dat deelnemers niet letterlijk onthouden hoe taken voltooid moeten worden, wel hoe de strategieën moeten worden geoptimaliseerd. Dit meet je door dezelfde performantiemetriek op verschillende momenten (trials) te meten en te kijken hoe dit over de tijd heen verloopt (snelheid gaat vaak omhoog/tijdskosten omlaag)

Zelfgerapporteerde metrieken

Performantiemetrieken zijn heel goed voor wat-vragen, maar hoe beantwoord je dan waarom-vragen? Dat kan je doen door zelfgerapporteerde metrieken.